

Projet : Conception d'une Architecture Microservices pour une Plateforme de Livraison de Repas

Objectif

Concevoir et documenter une architecture microservices pour une plateforme de livraison de repas (similaire à Uber Eats, Deliveroo, etc.). L'accent est mis sur la **conception architecturale**, la justification des choix techniques et la mobilisation des concepts vus en cours.

Un **prototype** est demandé, mais comme **démonstration** des choix d'architecture (interactions, patterns), et non comme un produit complet : il sert à *prouver* que la conception fonctionne, pas à couvrir toutes les fonctionnalités métier.

Contexte Métier

La plateforme doit permettre aux utilisateurs (clients) de parcourir les menus de restaurants partenaires, de passer des commandes, de payer en ligne, et de suivre la livraison effectuée par des livreurs indépendants. Les restaurants doivent pouvoir gérer leurs menus et commandes, et les livreurs gérer leurs disponibilités et livraisons.

Exigences Fonctionnelles Principales

Le système devra, à terme, supporter les fonctionnalités suivantes (la conception doit en tenir compte, même si l'implémentation est partielle) :

1. Gestion des Restaurants:

- Inscription/Profil du restaurant
- Gestion des menus (plats, prix, descriptions, options)
- Gestion des horaires d'ouverture
- Acceptation/Refus des commandes
- Suivi des commandes en préparation

2. Gestion des Clients:

- Inscription/Authentification/Profil client
- Gestion des adresses de livraison
- Historique des commandes

3. Catalogue et Recherche:

- Consultation des restaurants par localisation, type de cuisine, etc.
- Recherche de plats spécifiques
- Affichage des menus détaillés

4. Gestion des Commandes:

- Création d'un panier

- Passage de commande
- Calcul du prix total (incluant livraison)
- Suivi de l'état de la commande (Reçue, En préparation, En livraison, Livrée, Annulée)

5. Gestion des Paiements:

- Intégration avec un système de paiement externe sécurisé
- Gestion des remboursements (partiels ou totaux)

6. Gestion des Livreurs:

- Inscription/Profil du livreur
- Gestion des disponibilités
- Acceptation des propositions de livraison
- Suivi de la localisation (simulé ou réel)

7. Gestion des Livraisons:

- Assignment d'un livreur disponible à une commande prête
- Suivi en temps réel de la livraison par le client
- Confirmation de livraison

8. Gestion des Évaluations:

- Évaluation du restaurant par le client après commande
- Évaluation du livreur par le client
- (Optionnel) Évaluation du client par le livreur/restaurant

9. Notifications:

- Notifications aux clients (confirmation commande, départ livraison, etc.)
- Notifications aux restaurants (nouvelle commande)
- Notifications aux livreurs (proposition de livraison)
- Canaux : Email, Push (simulé), SMS (simulé)

Exigences Techniques et Architecturales

Les étudiants devront concevoir une architecture microservices répondant aux exigences suivantes :

1. Découpage en Microservices:

- Identifier les Bounded Contexts pertinents (ex: Commande, Paiement, Restaurant, Livraison, Client, Notification...).
- Proposer un découpage logique en microservices, en justifiant les choix (DDD, cohésion, couplage).

2. Communication Inter-Services:

- Définir les modes de communication (Synchrone/Asynchrone) entre les services pour différents scénarios.
- Choisir les protocoles appropriés (REST, gRPC, événements via Broker).

3. Design des API:

- Définir le contrat des API exposées par au moins les services clés : ressources/endpoints, formats, codes de retour.

- Documenter ces contrats (OpenAPI/Swagger pour REST, ou schéma pour GraphQL).
- Aborder le versionnement et la rétrocompatibilité des API.

4. Gestion des Données:

- Proposer un modèle de données par service (pas de BDD partagée).
- Définir les stratégies de gestion de la cohérence (cohérence éventuelle, SAGA...).

5. Transactions Distribuées:

- Identifier au moins un processus métier nécessitant une transaction distribuée (ex: passage de commande complet).
- Concevoir une solution basée sur le pattern SAGA (Orchestration ou Chorégraphie).

6. Résilience:

- Identifier les points de défaillance potentiels.
- Proposer l'implémentation d'au moins un pattern de résilience (ex: Circuit Breaker, Retry, Timeout, Fallback).

7. API Gateway:

- Définir le rôle d'une API Gateway (ou BFF) dans l'architecture.

8. Infrastructure:

- Proposer une infrastructure de déploiement simple basée sur des conteneurs (Docker Compose).

Extensions Optionnelles (Bonus)

Ces extensions ne sont **pas exigées** et ne doivent pas se faire au détriment du tronc commun. Elles permettent aux groupes qui le souhaitent d'aller plus loin et d'être valorisés (voir grille, points bonus plafonnés). Privilégier la **qualité** d'une extension bien justifiée à la quantité.

- **Messagerie événementielle avancée (Kafka)** : topics, partitions, consumer groups, garanties de livraison — en lien avec le chapitre 8.
- **CQRS / Event Sourcing** : séparation lecture/écriture, projections, journal d'événements — en lien avec le chapitre 9.
- **GraphQL** : en complément ou alternative justifiée à REST pour un besoin précis (ex: agrégation côté client) — chapitre 5.
- **Cache** : stratégie de mise en cache (ex: Redis) avec gestion de l'invalidation — chapitre 6.

Livrables Attendus

1. Documentation d'Architecture (Format Markdown):

- Description générale de l'architecture proposée.
- Analyse du domaine et justification du découpage (Bounded Contexts, microservices).
- Description de chaque microservice identifié (responsabilités, modèle de données principal).
- Description des patterns de communication choisis entre les services clés.
- **Contrats d'API** des services clés (OpenAPI/Swagger ou schéma GraphQL).
- Description de la stratégie de gestion de la cohérence et de la SAGA choisie pour un processus critique.
- Description du pattern de résilience implémenté.
- Architecture Decision Records (ADRs) pour les choix architecturaux majeurs.

2. Diagrammes:

- Diagramme de contexte système (niveau 1).

- Diagramme de conteneurs (niveau 2) montrant les microservices et leurs dépendances principales (BDD, Broker...).
- Diagramme(s) de séquence pour illustrer la SAGA et/ou le pattern de résilience.
- (Optionnel) Modèles de données simplifiés pour les services clés.
- *Utiliser un outil de diagramme au choix (Mermaid, draw.io, Lucidchart, etc.) et inclure les images ou liens dans la documentation.*

3. Prototype Minimal:

- Code source (langage au choix parmi Python, Node.js, Go, C#...) pour quelques services clés (au moins 3) avec des APIs mockées (pas besoin de logique métier complexe ni de BDD réelle). L'objectif est de démontrer la structure et les interactions.
- Fichier `docker-compose.yml` permettant de lancer l'infrastructure de base (services mockés, broker de messages si pertinent, API Gateway si pertinent).
- Démonstration claire dans le code de l'implémentation du pattern de résilience choisi et/ou de la SAGA.

4. Présentation Orale (15 minutes maximum par groupe):

- Support de présentation (slides).
- Explication claire de l'architecture et des choix de conception.
- Démonstration du prototype et des patterns implémentés.
- Session de questions/réponses.

Modalités de Réalisation et de Rendu

- **Constitution des groupes** : individuel ou groupe de **3 étudiants maximum**.
- **Charge de travail estimée** : environ **15 heures par étudiant**.
- **Rendu** : dépôt sur **MyGES** (et, le cas échéant, lien vers un dépôt Git) regroupant la documentation, les diagrammes, le code du prototype et le support de présentation.
- **Date limite de rendu** : à vérifier sur la plateforme MyGES.
- **Soutenance** : présentation orale de **15 minutes maximum** par groupe, suivie d'une session de questions/réponses.

Technologies Suggérées (non imposées, sauf mention contraire)

- **Langages**: Python (Flask/FastAPI), Node.js (Express), Go, C# (.NET Core).
- **Conteneurisation**: Docker, Docker Compose.
- **API Gateway**: Kong, Ocelot (.NET), Express Gateway, ou mock simple.
- **Broker de Messages**: RabbitMQ (plus simple pour un projet court), Kafka (si les étudiants veulent explorer).
- **Cache**: Redis (optionnel).
- **BDD**: PostgreSQL, MongoDB, ou BDD en mémoire pour les mocks.
- **APIs**: REST (OpenAPI/Swagger pour la doc), GraphQL (optionnel).

Critères d'Évaluation

L'évaluation porte sur 100 points, répartis entre conception architecturale, prototype, documentation et présentation orale, avec des points bonus optionnels (plafonnés à 100/100). L'accent est mis sur la compréhension des concepts, la pertinence des choix architecturaux, la clarté de la documentation et de la présentation.